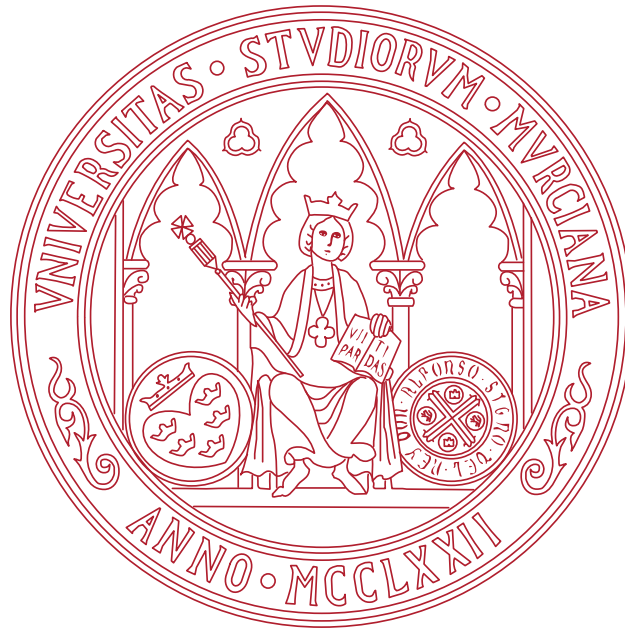




UNIVERSIDAD DE MURCIA

TRABAJO FIN DE GRADO

Impacto de los sistemas de memoria en grandes modelos de lenguaje

Estudiante:

Javier Patricio LUJÁN ROMERO
jp.lujanromero@um.es

Tutor:

Eduardo MARTÍNEZ GRACIÁ
edumart@um.es

Enero 2026

A mi familia por su apoyo incondicional desde el inicio.

Mi madre y su ternura.

Mi padre y su sacrificio.

Índice general

Resumen	IV
Extended abstract	v
1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Estructura del documento	2
2. Estado del arte	3
2.1. Grandes modelos de lenguaje	3
2.2. La ventana de contexto y sus problemas	5
2.3. Sistemas de memoria	7
2.3.1. Escenarios de uso	8
2.3.2. Mem0	9
2.4. LongMemEval	10
2.5. Métricas de evaluación	12
2.5.1. Métricas compartidas	12
2.5.2. Métricas específicas del uso de un sistema de memoria	13
3. Análisis de objetivos y metodología	15
3.1. Objetivos	15
3.2. Herramientas y tecnologías	16
3.2.1. Entorno de Programación	16
3.2.2. Entorno de desarrollo	17
3.2.3. Modelos de lenguaje	17
3.3. Metodología de trabajo	18
4. Diseño y resolución	20
4.1. Pipeline de evaluación	20
4.1.1. Implementación concreta	21
4.2. Evaluación con LongMemEval-Oracle	23
4.3. Evaluación con LongMemEval-S	29
4.4. Posibles mejoras del sistema de memoria	29
4.5. C02	29

5. Conclusiones y vías futuras	30
Bibliografía	32

Índice de figuras

2.1.	Arquitectura de un Transformer. Extraído de [13].	4
2.2.	Visualización del proceso de tokenización. Fuente: Elaboración propia utilizando la herramienta Tokenizer [7] de OpenAI.	5
2.3.	Evolución de la ventana de contexto en los últimos años. Datos extraídos de OpenAI y MetaAI.	6
2.4.	Visualización de los dos escenarios de uso. Fuente: Elaboración propia. . .	9
2.5.	Pipeline de adición de memorias en Mem0. Extraído de [2].	11
2.6.	Pipeline de búsqueda de Mem0. Extraído de [2].	11
4.1.	Esquema del pipeline de evaluación diseñado para este proyecto.	21
4.2.	Comparativa de la métrica de precisión (LLM-Judge).	24
4.3.	Comparativa de la métrica de latencia.	26
4.4.	Comparativa de la métrica de uso de tokens en memoria.	28
4.5.	Tiempo de procesamiento del contexto en Mem0 (LongMemEval-Oracle). .	29

Resumen

En este proyecto se plantea el problema de...

Extended abstract

This project faces the problem of...

Introducción

1.1. Motivación

En los últimos años la IA, en particular los grandes modelos de lenguaje – LLMs por sus siglas en inglés– han supuesto una revolución y un cambio de paradigma ya no solo dentro de la industria del Machine Learning, sino en todos los ámbitos de la sociedad. Las aplicaciones de chat basadas en estos grandes modelos de texto, como ChatGPT o Google Gemini, se han convertido en herramientas principales para millones de usuarios en todo el mundo realizando una inmensidad de tareas distintas.

Estas tareas tienen diferentes niveles de complejidad, desde tareas simples que se resuelven con una respuesta directa, a tareas más complejas en las que el propio modelo debe replantear los conceptos con los que está trabajando y cómo los ha de usar para llegar a la solución.

Dentro de estas tareas complejas, se pueden plantear dos situaciones: una en la que el modelo dispone de toda la información necesaria para resolver la tarea en el prompt, y la complejidad radica en el proceso deductivo que el modelo debe realizar para llevarla a cabo exitosamente, como por ejemplo la resolución de problemas matemáticos avanzados. Por otro lado, tenemos el caso en el que la resolución exitosa de la tarea requiere de otra información adicional que no se encuentra en el prompt; como ejemplo de este tipo de tarea tenemos la modificación de un código fuente ajustándose a los estándares del proyecto en el que se encuentra.

Es dentro de este segundo tipo de tareas donde ha proliferado el uso de los conocidos agentes de IA: sistemas autónomos capaces de percibir su entorno, razonar sobre posibles cursos de acción y tomar y ejecutar decisiones [9]. Estos agentes tienen como base un LLM principal, el cual realiza el razonamiento, y se le dota de un conjunto diverso de herramientas que le permiten interactuar con su entorno. Ejemplos de estas herramientas son: navegadores web, RAG en bases de datos, editores de código e incluso otros modelos de IA especializados en diferentes tareas.

Pero para el desarrollo de este trabajo, nos vamos a centrar en un tipo de herramienta en concreto: los sistemas de memoria [2]. Estos sistemas permiten a los agentes almacenar y recuperar información relevante a lo largo del tiempo, mejorando su capacidad para manejar tareas que requieren conocimiento a largo plazo y reduciendo la necesidad de incluir grandes cantidades de texto en el prompt. De esta manera se construyen agentes más eficientes tanto a nivel computacional como en el uso de tokens, que por ende son más económicos y generan un menor impacto ambiental.

1.2. Objetivos

El objetivo principal del trabajo es realizar un estudio sobre el impacto que tienen los sistemas de memoria en el rendimiento de los LLMs en distintos términos: latencia, precisión y consumo de tokens.

Con el fin de realizar este primer objetivo se ha creado un pipeline modular, que permite integrar diferentes sistemas de memoria para LLMs, facilitando la experimentación y evaluación de estos sistemas en diversas tareas y datasets. Se cumple de esta manera el segundo objetivo, que consiste en diseñar un sistema accesible que permita evaluar diferentes sistemas de memoria en LLMs de manera sencilla y adaptable.

1.3. Estructura del documento

El trabajo se ha estructurado de acuerdo con las indicaciones dadas en la guía docente de la asignatura correspondiente al Trabajo de Fin de Grado. Por tanto, el resto del documento se organiza de la siguiente manera:

- En el capítulo 2 se realiza un estudio del estado del arte relativo a los conceptos tratados en este trabajo, abarcando desde la arquitectura detrás de los LLMs hasta los sistemas de memoria.
- En el capítulo 3 se describen de manera detallada los objetivos del trabajo, se introducen las herramientas y tecnologías empleadas, y se explica la metodología de trabajo seguida.
- En el capítulo 4, diseño y resolución, se expone y explica la implementación realizada del pipeline que permite realizar la evaluación de los sistemas de memoria.
- Finalmente, en el capítulo 5 se exponen los resultados obtenidos tras la evaluación de un sistema de memoria, sacando conclusiones de los mismos y proponiendo vías de mejora a futuro.

Estado del arte

En este capítulo se realiza un estudio del estado del arte relativo a los conceptos sobre los que trata el trabajo. Se inicia con una introducción a los grandes modelos de lenguaje en la sección 2.1, permitiendo entender la arquitectura que tienen detrás y sus principales características. En la siguiente sección, la 2.2, se expone el concepto de la ventana de contexto y los principales problemas asociados a su uso. A continuación se presentan en la sección 2.3 los sistemas de memoria como posible solución a los problemas mencionados anteriormente. Posteriormente, en la sección 2.4 se describe LongMemEval, el dataset elegido como benchmark para evaluar el rendimiento de los sistemas de memoria. Finalmente, en la sección 2.5 se describen las métricas que se han usado para evaluar la diferencia de rendimiento entre el uso exclusivo de la ventana de contexto y el uso de un sistema de memoria.

2.1. Grandes modelos de lenguaje

Como se ha expuesto en la introducción, la IA y los grandes modelos de lenguaje (LLMs) son, cada vez más, un pilar fundamental en nuestra sociedad. Por tanto, es necesario entender, con cierto grado de profundidad, qué significa cada uno de estos conceptos y en qué se diferencian.

Por un lado, el concepto más general de todos es el de inteligencia artificial (IA). La IA es un campo de estudio de la informática cuyo objetivo es crear máquinas inteligentes capaces de mejorar de manera autónoma explorando y aprendiendo del mundo real [16]. Dentro de este campo encontramos el Aprendizaje Computacional, o Machine Learning, que es la rama que se encarga de la construcción de algoritmos que, a partir de un conjunto de datos de entrenamiento, son capaces de aprender patrones, lo que les permite posteriormente realizar predicciones o tomar decisiones sobre datos nuevos que no han visto antes; es decir, les permite generalizar [12]. Finalmente, es dentro del Machine Learning donde nos encontramos con el Aprendizaje Profundo, o Deep Learning,

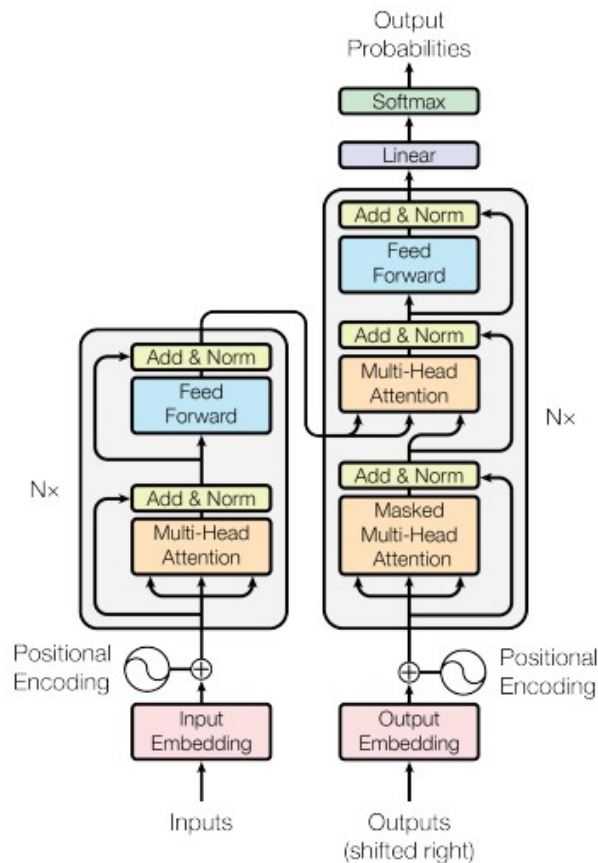


Figura 2.1: Arquitectura de un Transformer. Extraído de [13].

que se acerca por fin a la concepción generalizada de la IA.

Este aprendizaje profundo se basa en redes neuronales: un modelo computacional inspirado en las neuronas biológicas del cerebro humano, que se comunican a través de señales eléctricas [12]. Esto se traduce en una serie de capas conectadas entre sí de perceptrones – un modelo matemático de neurona. La profundidad radica en el número de capas ocultas, que se encuentran entre la capa de entrada y la de salida; esto les permite aprender representaciones jerárquicas y complejas de los datos, mediante el uso de algún algoritmo que ajusta la fuerza de las conexiones entre las neuronas individuales de capas adyacentes, lo que se conoce como pesos [12].

En un principio, entrenar estos modelos de Deep Learning requería recursos computacionales significativos y bastante tiempo. Sin embargo, desde hace unos años se ha desarrollado una nueva arquitectura de red neuronal, los denominados **Transformers**, que ha revolucionado el campo del Deep Learning, especialmente en el procesamiento del lenguaje natural (NLP) [13]. Es gracias a esta arquitectura que se han podido crear los LLMs (Large Language Models), llamados así por la inmensa cantidad de datos con los que son entrenados y por su elevado número de parámetros, los valores numéricos ajustables que incluyen, entre otros, los pesos de las conexiones entre las neuronas.

Cabe, por tanto, finalizar esta sección con una breve explicación sobre esta arquitec-

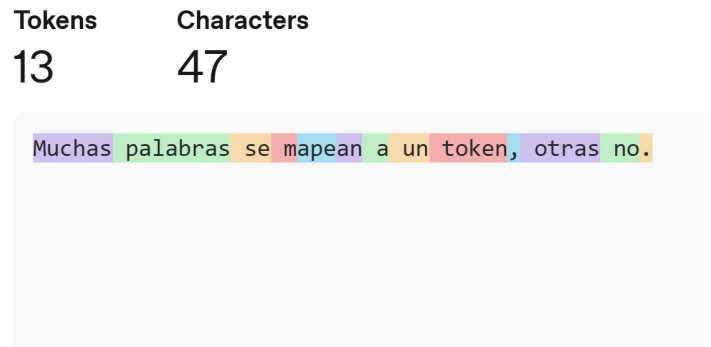


Figura 2.2: Visualización del proceso de tokenización. Fuente: Elaboración propia utilizando la herramienta Tokenizer [7] de OpenAI.

tura. Un Transformer es una arquitectura de red neuronal basada en un mecanismo de atención, que permite al modelo enfocarse en diferentes partes de la entrada de manera adaptativa, siendo por tanto altamente efectiva a la hora de procesar y generar secuencias de datos, como texto, audio o video; y que es además altamente paralelizable.

Los Transformers se componen principalmente de dos partes: el codificador (encoder) y el decodificador (decoder), la parte izquierda y derecha de la figura 2.1 respectivamente [13]. El codificador toma la secuencia de entrada previamente convertida en embeddings, representaciones numéricas, y la transforma en una representación interna que captura tanto la palabra como su contexto, mientras que el decodificador utiliza esta representación para generar la secuencia de salida, palabra por palabra de manera auto-regresiva, es decir, utilizando la salida generada previamente para generar la siguiente. Ambos componentes están formados por múltiples capas de atención y redes neuronales feedforward.

2.2. La ventana de contexto y sus problemas

En la sección anterior se ha explicado brevemente la arquitectura detrás de los Transformers; sin embargo, no se ha detallado cómo se introduce la secuencia de entrada en estos modelos. Es importante destacar que los modelos no trabajan con texto plano, sino que, como se ha indicado, toman como entrada *embeddings*. Estos *embeddings* son vectores de valores continuos que representan tanto a las palabras como su relación semántica con otras, y son las unidades matemáticas con las que opera el modelo [6].

Para llegar a generar estos *embeddings*, existe un paso previo fundamental: la **tokenización**. En este proceso, las palabras, caracteres individuales o subconjuntos de caracteres, son divididos en unidades mínimas de información llamadas *tokens* [17], tal como se muestra en la figura 2.2. Finalmente, cada uno de estos *tokens* es convertido en un primer *embedding* base para ser procesado por el modelo.

Los tokens constituyen, por tanto, la unidad básica de información con la que se introduce el texto en los modelos de lenguaje. Y al ser la base sobre la que se calcula la representación interna del texto, se utilizan como métrica estándar para establecer los

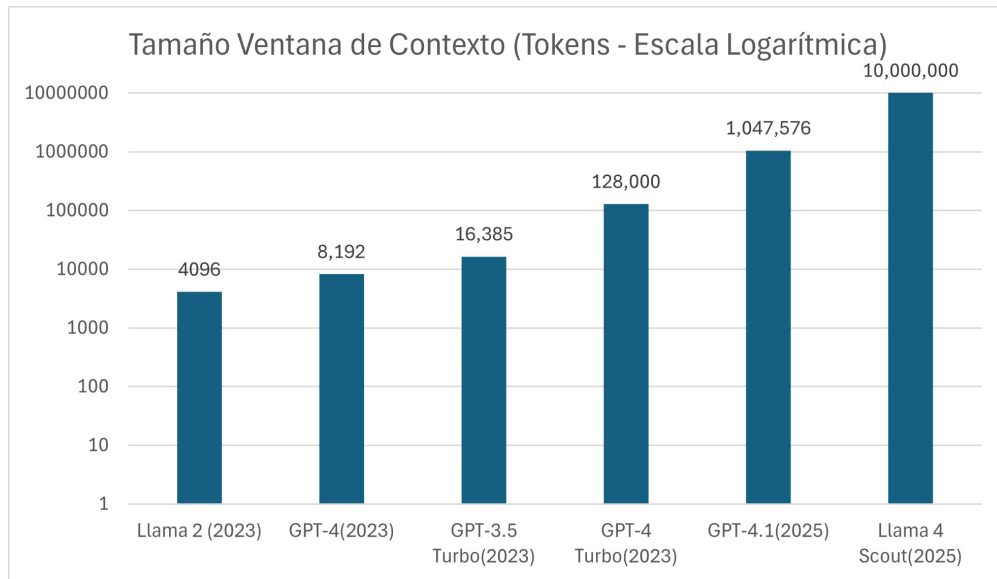


Figura 2.3: Evolución de la ventana de contexto en los últimos años. Datos extraídos de OpenAI y MetaAI.

límites de capacidad en la entrada de datos. Este límite se denomina de manera formal *ventana de contexto*, y se refiere a la cantidad máxima de tokens que un modelo puede procesar en una sola instancia.

La longitud de la ventana de contexto ha sido un componente en el que se ha puesto especial énfasis y ha sufrido un aumento drástico en los últimos años (véase la figura 2.3) pasando de unos pocos miles de tokens a más de un millón en algunos modelos recientes [1]. Este aumento ha sido posible gracias a avances en la arquitectura de los modelos, como la introducción de mecanismos de atención más eficientes, lo que ha permitido a los modelos manejar contextos más largos sin una degradación significativa del rendimiento [1].

Pero, aunque como se acaba de ver, la tendencia actual de la industria es un progresivo aumento de la ventana de contexto, no se debe confiar exclusivamente en esta como único modelo para la gestión de la información, pues esta solución no está exenta de problemas, entre los que destacan los siguientes:

1. Un primer problema que tiene la ventana de contexto es que, como indica su propia definición, es, de antemano, un límite fijado [2]. Esto implica que, aunque se pueda aumentar la ventana de contexto, siempre existirá un límite máximo de tokens que el modelo pueda procesar, lo que puede resultar insuficiente para tareas que requieren trabajar con grandes cantidades de información o con conocimiento a largo plazo.
2. Por otro lado, aunque el contexto con el que se está trabajando quepa dentro de la ventana, procesar grandes cantidades de texto tiene un coste computacional significativo, lo que se traduce en un coste económico elevado [5]. Esto hace que, aunque se pueda aumentar la ventana de contexto, no sea una solución viable a largo plazo ni asumible en todas las aplicaciones, pues el coste de procesar grandes cantidades de texto deja de ser económicamente viable.

3. Además, aun ignorando tanto el coste computacional como el límite de tokens, la ventana de contexto tiene otra desventaja crucial: el rendimiento de los modelos de lenguaje, es decir, la capacidad de procesar y responder correctamente a preguntas sobre una entrada, tiende a degradarse a medida que aumenta su tamaño [3], siendo aún más pronunciada esta degradación cuando en el contexto se incluye información irrelevante para la tarea específica que se está realizando, lo que puede resultar en una disminución significativa del rendimiento del modelo [11].
4. Finalmente, el último desafío que se presenta es en la propia naturaleza del aprendizaje. La ventana de contexto permite retener información con la que trabajar, similar a la memoria de trabajo humana, pero es incapaz de procesar la información a lo largo del tiempo, e incluso de diferentes conversaciones y chats [8]. Esto limita las capacidades de un LLM para generalizar y aprender de la experiencia, pues no puede retener información a largo plazo, lo que se traduce en una falta de continuidad y coherencia en las conversaciones, y en una incapacidad para aprender de la experiencia pasada.

2.3. Sistemas de memoria

Como solución a estos problemas, se han propuesto diferentes enfoques, entre los que destacan los basados en memoria –*la habilidad para almacenar, actualizar y recuperar conocimiento* [15]. El objetivo de los mismos es reducir la necesidad de procesar grandes cantidades de texto, economizando así recursos computacionales, y a su vez, permitiendo mejorar el rendimiento en tareas para las que la ventana de contexto es insuficiente.

Para entender estos sistemas de memoria es necesario responder a una serie de preguntas clave: ¿de dónde viene la información que se va a almacenar en la memoria? ¿qué se debe hacer con esta información? y ¿cómo se pueden almacenar dichas memorias?

En cuanto a las fuentes de la información, Zhang et al. [16] proponen tres categorías principales: la información de la tarea que se está realizando, la información de tareas que ya se han realizado, e información externa obtenida de herramientas. Mientras que la primera es la más relevante para la tarea actual, y la tercera permite expandir el conocimiento del modelo más allá de sus barreras iniciales, la segunda es crucial para permitir que el modelo aprenda de la experiencia, lo que es fundamental para lograr una memoria a largo plazo.

Una vez obtenida esta información, es necesario decidir qué hacer con ella y como llevarla de información a memoria. Para esto, se pueden identificar tres procesos clave: la proyección de observaciones directas a memorias concisas e informativas; la consolidación de memorias, procesando, organizando y asentando información para mejorar su calidad y relevancia; y la recuperación de memorias, que consiste en recuperar información importante de la memoria para apoyar la toma de decisiones y la generación de respuestas [16].

Finalmente, queda la cuestión de cómo almacenar las memorias para poder realizar las tres operaciones anteriores de manera eficiente. Para esto, se pueden clasificar los

sistemas de memoria en dos grandes categorías: memoria en forma textual y memoria en forma paramétrica [16].

La primera consiste en guardar las memorias en forma de texto, ya sea estructurado o no, e introducirlo en el contexto del agente, lo que permite una mayor flexibilidad y adaptabilidad a diferentes tareas, aunque a expensas de la eficiencia computacional y el sobrecoste de procesamiento. Por otro lado, la forma paramétrica consiste en guardar las memorias, mediante la actualización de los parámetros del modelo, lo que permite una *recuperación* más rápida y eficiente, pero a expensas de la interpretabilidad y la flexibilidad [16].

Debido a las ventajas que ofrece la memoria en forma textual, es un enfoque que ha sido ampliamente probado en diferentes sistemas de memoria como: MemoryBank [18], un sistema de memoria que imita a los humanos centrándose en almacenar la información necesaria para entender la personalidad del usuario; Zep [10], que introduce una capa de memoria basada en un grafo de tres niveles que abarca desde los datos en crudo hasta abstracciones semánticas; o CLIN [4], un sistema persistente y dinámico de memoria textual basado en frases que representan abstracciones causales obtenidas de la interacción del agente con el entorno a la hora de aprender a realizar diferentes tareas.

2.3.1. Escenarios de uso

Una vez introducidos los sistemas de memoria, vamos a proceder a describir los dos escenarios de uso que se van a comparar en este trabajo: el uso exclusivo de la ventana de contexto, y el uso de un sistema de memoria.

En ambos escenarios se asume que se esta trabajando con un chat entre un usuario y un asistente de IA, y se evalúa la capacidad que tiene el asistente para responder a una pregunta concreta, cuya respuesta se encuentra en una conversación previa.

En el caso del **uso exclusivo de la ventana de contexto**, al asistente se le introduce como contexto toda la conversación previa, incluyendo tanto la información relevante para responder a la pregunta como la irrelevante, y se le pide que responda a la pregunta utilizando toda esta información.

En este escenario, se apreciarán claramente los problemas asociados a la ventana de contexto (2.2), permitiendo evaluar el rendimiento del modelo en escenarios donde la cantidad de información supera la capacidad de la ventana de contexto, analizar la degradación bajo la presencia de información irrelevante, y evaluar el coste computacional asociado a procesar grandes cantidades de texto.

Por otro lado, en el caso del **uso de un sistema de memoria** se debe separar el proceso de respuesta en dos fases. Una primera fase que realiza la creación de memorias de la conversación, en la que se debe simular el proceso que hubiera seguido el sistema de memoria para crear las memorias durante la conversación, introduciendo al sistema de memoria la conversación en pares usuario-asistente, y obteniendo así las memorias que el sistema hubiera creado. Y una segunda fase de recuperación de memorias, donde se introduce la pregunta al sistema de memoria, y este devuelve las memorias relevantes

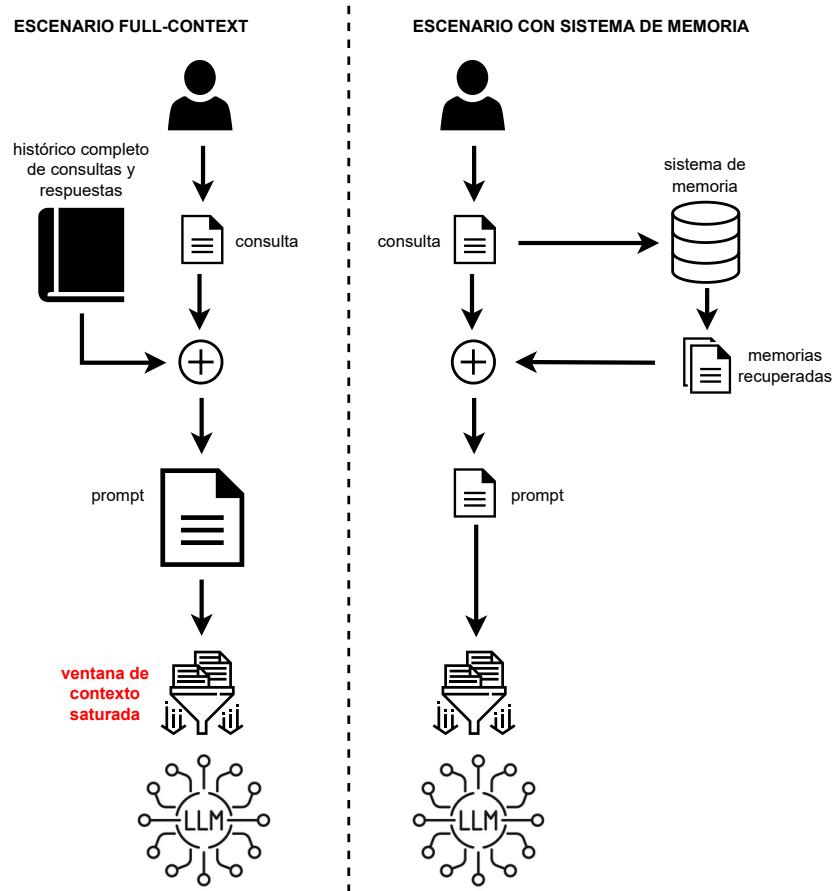


Figura 2.4: Visualización de los dos escenarios de uso. Fuente: Elaboración propia.

para responder a la pregunta, que serán introducidas en el contexto del modelo para que este pueda responder a la pregunta utilizando únicamente esta información relevante.

En este escenario se evaluará entonces la capacidad del sistema de memoria para: 1º crear las memorias relevantes a lo largo de la conversación, 2º recuperar las memorias relevantes para responder a la pregunta, y 3º mejorar el rendimiento del modelo al reducir la cantidad de información irrelevante que se introduce en el contexto.

2.3.2. Mem0

Entre los diferentes sistemas de memoria que hay en el mercado, Mem0 [2] ha sido elegido como base para este trabajo, permitiendo realizar la comparativa entre la ventana de contexto y el uso de un sistema de memoria. Esta decisión se ha tomado así por ser Mem0 un ejemplo canónico de sistema de memoria, por su facilidad de uso e integración, y por ser un sistema actual y con una versión open source continuamente actualizada.

Es por ende necesario entender el funcionamiento de Mem0 para poder comprender la relación entre la mejora que supone y los posibles factores negativos que se pueden

asociar a este.

Mem0 es un sistema de memoria textual, pero su componente principal es una base de datos vectorial en la que se almacenarán embeddings de las memorias conforme se vayan generando, permitiendo una búsqueda y recuperación más eficiente. Mem0 actúa de wrapper sobre esta base de datos, permitiendo usar cualquier tipo de BBDD vectorial que se desee, lo que aporta una gran flexibilidad y, en caso de que sea necesario, privacidad.

Su comportamiento se divide en dos fases principales: la extracción de posibles memorias de la conversación, y la actualización de la base de datos:

La **extracción**, como se aprecia en la figura 2.5, se realiza en cada iteración del chat, tomando como entrada el último par de mensajes –usuario y respuesta del modelo; una secuencia con los últimos m mensajes, siendo este número un hiperparámetro; y finalmente un resumen de la conversación generado de manera asíncrona en la base de datos. Toda esta información se introduce en un prompt y se envía a un LLM, previamente configurado por el operador del sistema, con el fin de que este extraiga memorias candidatas para añadir a la base de datos.

Una vez realizada esta fase se procede con la de **actualización**. En esta se realiza el mismo procedimiento para cada memoria candidata extraída en el paso anterior, este consiste en realizar una búsqueda semántica en la base de datos tomando como prompt la memoria candidata. Entonces, usando tanto la memoria candidata como las memorias obtenidas en la consulta, se realiza una segunda llamada a un LLM, el cual debe decidir entre cuatro posibles acciones: *ADD*, *UPDATE*, *DELETE* y *NOOP*. Que significan:

- **NOOP**: No hacer nada, la memoria candidata no aporta nada nuevo a la base de datos.
- **UPDATE**: Actualizar alguna de las memorias obtenidas en la consulta con la información de la memoria candidata.
- **ADD**: Añadir la memoria candidata como una nueva memoria en la base de datos.
- **DELETE**: Borrar alguna de las memorias obtenidas en la consulta, pues ya no es relevante.

En la figura 2.5 se puede observar la pipeline de creación de memorias de Mem0.

Finalmente, para la recuperación de memorias, Mem0 vuelve a hacer uso de un LLM. Este reescribe el mensaje del usuario y realiza una búsqueda semántica en la base de datos, devolviendo las memorias más relevantes para ser introducidas en el contexto del modelo (véase la figura 2.6).

2.4. LongMemEval

Para evaluar el rendimiento que ofrece el uso de Mem0 frente al empleo exclusivo de la ventana de contexto, se ha elegido usar como benchmark el dataset LongMemEval

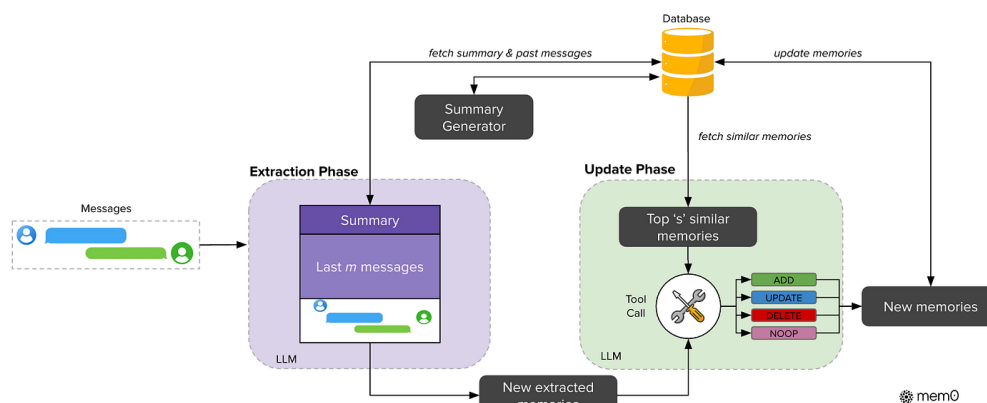


Figura 2.5: Pipeline de adición de memorias en Mem0. Extraído de [2].

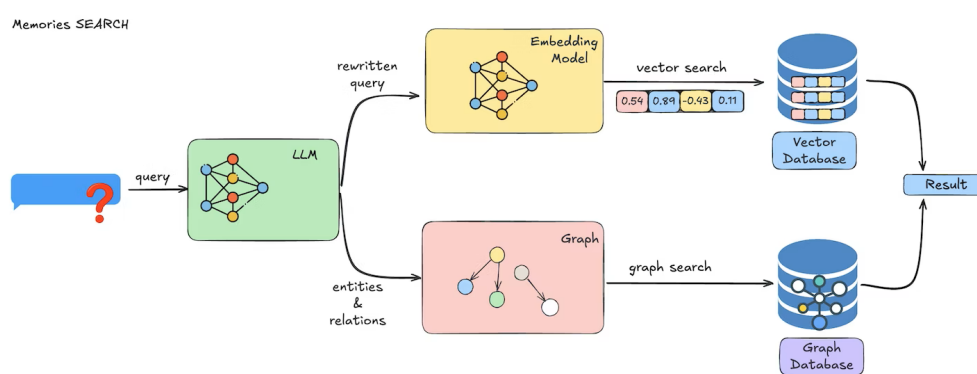


Figura 2.6: Pipeline de búsqueda de Mem0. Extraído de [2].

[14] creado específicamente para evaluar las capacidades de memoria a largo plazo de los asistentes de IA.

El dataset se compone de un conjunto de 500 pares pregunta-respuesta, cada uno de ellos acompañado de un conjunto de sesiones de chat –una serie de interacciones o turnos usuario-asistente. En ellas se encuentra tanto la información necesaria para responder a la pregunta como conversación adicional irrelevante.

LongMemEval cuenta con tres versiones distintas, diferenciadas por la cantidad de información irrelevante que se introduce en el contexto, lo que permite evaluar el rendimiento de los modelos en diferentes escenarios. Estas son:

- **LongMemEval-Oracle:** versión baseline que solo contiene sesiones de chat con información relevante para la pregunta.
- **LongMemEval-S:** versión que introduce una cantidad moderada de mensajes irrelevantes, con aproximadamente 115K tokens por pregunta.
- **LongMemEval-M:** versión extendida con una gran cantidad de mensajes irrelevantes, llegando al 1.5M de tokens para cada problema.

De ellas solo se han usado las dos primeras, pues la generación de memorias es un proceso lento y el número de tokens de la versión M hace que el coste computacional sea demasiado elevado, lo que no es viable para este trabajo.

En lo que corresponde a las preguntas en sí, estas se dividen en siete tipos distintos y buscan evaluar cinco aspectos claves en el desarrollo de la memoria a largo plazo. Dichas habilidades y sus correspondientes tipos de preguntas son:

1. **Information Extraction:** la habilidad para extraer información relevante de una conversación larga. Las preguntas asociadas a esta habilidad son, por un lado *single-session-user* y *single-session-assistant*, que buscan evaluar la capacidad de memorizar información relevante de los mensajes del usuario o del asistente dentro de una sesión de chat; y por otro lado, *single-session-preference* donde se requiere memorizar o aprender preferencias del usuario.
2. **Multi-session Reasoning:** la habilidad para integrar información a lo largo de múltiples sesiones de chat y responder a preguntas complejas que requieren razonamiento inter-sesión. Las preguntas asociadas a esta habilidad son *multi-session* que evalúan la habilidad para agregar información a lo largo de múltiples sesiones de chat.
3. **Knowledge Updates:** la habilidad para reconocer cambios en la información y actualizar el conocimiento almacenado en consecuencia. Las preguntas asociadas a esta habilidad son *knowledge-update* que buscan evaluar la capacidad de actualizar el conocimiento almacenado en la memoria a largo plazo.
4. **Temporal Reasoning:** la habilidad para ser consciente de los aspectos temporales de la información. Sus preguntas asociadas son homónimas a la habilidad, es decir, *temporal-reasoning*, y buscan evaluar la capacidad de entender y razonar sobre la información temporal.
5. **Abstention:** la habilidad para identificar cuándo no se tiene suficiente información para responder a una pregunta y contestar indicándolo. Las preguntas asociadas a esta habilidad son *abstention*, y no son preguntas específicas sino que hay 30 preguntas diseminadas a lo largo de los otros seis tipos.

2.5. Métricas de evaluación

En esta sección se describen las diferentes métricas que se han usado para evaluar los distintos aspectos a tener en cuenta a la hora de comparar el uso exclusivo de la ventana de contexto frente al uso de un sistema de memoria. Estas métricas se han elegido para evaluar tanto el rendimiento del modelo a la hora de responder a las preguntas, como el coste computacional asociado a cada uno de los escenarios.

2.5.1. Métricas compartidas

En esta sección vamos a describir las métricas que se han evaluado en ambos escenarios:

- **Precisión:** métrica clave para evaluar el rendimiento del modelo a la hora de responder a las preguntas. Se calcula como el número de respuestas correctas dividido por el número total de preguntas, y se expresa como un porcentaje. Para poder determinar si una respuesta es correcta o no, se ha utilizado un modelo de lenguaje como juez, al que se le introduce la pregunta, la respuesta del modelo y la respuesta correcta proporcionada en el dataset.
- **Latencia:** mide el tiempo que tarda el modelo en generar una respuesta a una pregunta, desde el momento en que se introduce la pregunta hasta que se obtiene la respuesta. En el caso de los sistemas de memoria la latencia incluye el tiempo requerido para recuperar las memorias pero no el que se tarda en procesar la conversación para generar las memorias. Este tiempo se asume que ocurre de manera asíncrona durante la conversación, y no es un coste adicional a la hora de responder a la pregunta.
- **Tokens del prompt de entrada:** como su nombre indica esta métrica mide el número de tokens que tiene el prompt utilizado que se introduce al modelo para generar la respuesta a una pregunta. Esta métrica es importante pues el número de tokens del prompt afecta directamente al coste computacional de generar una respuesta, y a su vez, a la latencia.
- **Coste de CO2:** esta métrica mide el impacto ambiental asociado al uso de los modelos de lenguaje y los sistemas de memoria, calculando la cantidad de dióxido de carbono emitida durante todo el proceso de generación de respuestas. Su cálculo específico se explica en su sección correspondiente, pero es importante destacar que esta métrica es crucial para evaluar la sostenibilidad de cada uno de los enfoques, especialmente teniendo en cuenta el aumento del uso de los modelos de lenguaje y su impacto ambiental asociado.

2.5.2. Métricas específicas del uso de un sistema de memoria

En el caso del uso de un sistema de memoria, se han evaluado además las siguientes métricas específicas:

- **Tiempo de creación de memorias:** esta métrica mide el tiempo que tarda el sistema de memoria en procesar la conversación y generar las memorias correspondientes. Este tiempo se asume que ocurre de manera asíncrona durante la conversación, y no es un coste adicional a la hora de responder a la pregunta, pero es importante medirlo para evaluar la eficiencia del sistema de memoria pues se usará junto a la latencia y los tokens para calcular el coste de CO2.
- **F1:** esta medida se calcula como $F1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$ [16]. No obstante, la forma de calcular *precision* y *recall* depende de la unidad de análisis (tokens, memorias, sesiones o turnos). Esta métrica se usa ampliamente en la literatura, aunque su implementación varía según la tarea:

- *Token-level F1*: en este enfoque, la precision y el recall se calculan a nivel de tokens, normalmente comparando la respuesta generada por el modelo con la respuesta de referencia del dataset. Aunque es una metrica comun, puede no ser la mas adecuada para evaluar sistemas de memoria, ya que se centra en la respuesta final y en el solapamiento lexico, pudiendo penalizar parafrasis correctas [2].
- *F1 con LLM juez*: en este trabajo se adopta un enfoque alternativo en el que se evalua la calidad de las memorias recuperadas, no solo la respuesta final. Para ello, un LLM recibe la pregunta, la respuesta de referencia y las memorias recuperadas, y clasifica cada memoria como relevante o no relevante (estimando asi la precision). Ademias, el LLM estima la suficiencia del conjunto de memorias para responder la pregunta; a partir de esa suficiencia se aproxima el recall de forma heuristica. Este enfoque es adecuado para la tarea, ya que mide directamente la utilidad de la memoria recuperada.
- *F1 con etiquetas del dataset*: este enfoque tambien evalua las memorias recuperadas, pero usa anotaciones estructuradas del dataset en lugar de un LLM juez. En particular, permite medir la recuperacion a nivel de sesion (*session-level*) y a nivel de turno exacto (*turn-level*), comprobando si las memorias provienen de las partes relevantes de la conversacion. Es una medida complementaria a la anterior: aporta mayor objetividad estructural, aunque no evalua de forma directa la relevancia semantica del contenido recuperado.

Análisis de objetivos y metodología

Una vez tratado el estado del arte y presentados los conceptos teóricos que enmarcan a los LLMs y los sistemas de memoria, realizamos en este capítulo el análisis de los objetivos del mismo, estableciendo los elementos concretos que nos van a permitir llevar a cabo dichos objetivos así como la metodología de desarrollo.

3.1. Objetivos

El objetivo principal de este trabajo es analizar el impacto que tienen los sistemas de memoria en el rendimiento de los grandes modelos de lenguaje (LLMs) mediante la creación de un pipeline de evaluación. Con el fin de alcanzar este objetivo, se han establecido los siguientes subobjetivos:

- **Entender los sistemas de memoria:** Realizar un estudio teórico detallado sobre los sistemas de memoria en LLMs, examinando y comprendiendo la literatura actual sobre el tema y consultando implementaciones concretas.
- **Hacer uso de Mem0:** Entender y utilizar un sistema de memoria específico para integrar las funcionalidades de memoria en los LLMs dentro del flujo de trabajo.
- **Crear un entorno de evaluación:** Desarrollar un pipeline de evaluación que ensamble las herramientas necesarias para permitir medir el impacto de los sistemas de memoria en el rendimiento de los LLMs.
- **Establecer métricas de rendimiento:** Definir y establecer métricas específicas para evaluar el rendimiento de los LLMs con y sin la integración de sistemas de memoria, asegurando que estas métricas sean relevantes y adecuadas para el análisis.
- **Evaluar el rendimiento:** Realizar una evaluación exhaustiva del rendimiento de los LLMs con y sin la integración de sistemas de memoria, utilizando métricas específicas y el dataset Longmemeval-S.

- **Evaluar el impacto ambiental:** Analizar el impacto ambiental de la utilización de sistemas de memoria en los LLMs, considerando el consumo de recursos y la huella de carbono asociada.
- **Extraer conclusiones y plantear futuras líneas de investigación:** Analizar los resultados obtenidos en la evaluación para extraer conclusiones sobre el impacto de los sistemas de memoria en el rendimiento de los LLMs, y proponer posibles líneas de investigación futura basadas en los hallazgos obtenidos.

3.2. Herramientas y tecnologías

Para el desarrollo de este proyecto se ha hecho uso de un amplio abanico de herramientas y tecnologías, que han hecho posible la implementación y desarrollo de los objetivos planteados. A continuación se detallan las principales herramientas utilizadas:

3.2.1. Entorno de Programación

Se ha trabajado usando **Python** como lenguaje de programación principal, debido a la amplia disponibilidad de librerías que han permitido la implementación de los diferentes componentes del sistema. Todas estas librerías han sido gestionadas dentro de un entorno virtual creado con **venv**, lo que ha permitido mantener un entorno de desarrollo limpio y organizado. Entre ellas, destacan:

- **Mem0:** librería oficial de Mem0, permitiendo una integración sencilla de sus funcionalidades tanto en local como a través de su API.
- **Ollama:** librería oficial de Ollama, que ha permitido la utilización de diferentes modelos de lenguaje en el sistema desarrollado.
- **OpenAI:** librería oficial de OpenAI, que ha sido fundamental para la integración de los modelos de lenguaje de OpenAI en el sistema.
- **TikToken:** librería de OpenAI para el conteo de tokens, necesaria para la medición de la cantidad de tokens utilizados en las diferentes interacciones con los modelos de lenguaje.
- **Qdrant:** librería oficial de Qdrant, que ha permitido la integración de este sistema de base de datos vectorial en el proyecto.

Como entorno de desarrollo integrado (IDE), se ha utilizado **Visual Studio Code**, que ofrece una gran cantidad de extensiones y herramientas para facilitar el desarrollo en Python. Además, permite una integración fluida con entornos remotos a través de la extensión **Remote - SSH**, lo que ha sido fundamental para trabajar con la máquina proporcionada por la universidad.

3.2.2. Entorno de desarrollo

Para el desarrollo de este proyecto se han utilizado, de manera directa, tres máquinas diferentes.

En primer lugar, para el desarrollo de código y las pruebas iniciales se usó un **ordenador personal** con las siguientes características: procesador Intel Core i7-11370H de 11ª generación, 16 GB de RAM, y una tarjeta gráfica NVIDIA GeForce RTX 3060 Laptop, todo bajo un sistema operativo Windows 11. Este ordenador fue clave para el desarrollo inicial de proyecto, permitiendo la implementación y prueba con pequeños modelos de lenguaje.

Posteriormente, con el fin de poder trabajar con modelos de lenguaje más grandes y realizar pruebas más exhaustivas, se hizo uso de **Ollama en un servidor del grupo de investigación Sistemas Inteligentes y Telemática de la Universidad de Murcia**. Dicho servidor tiene las siguientes características: sistema operativo Debian GNU/Linux 12 (bookworm), procesador AMD EPYC 9554 a 3.095 GHz, dos GPUs NVIDIA GH100 y 515975 MiB de memoria RAM (aprox. 516 GB). Esta máquina permitió la ejecución de modelos de lenguaje más complejos y la realización de pruebas a mayor escala.

Además, para poder alojar la base de datos vectorial empleada para Mem0, y poder ejecutar pruebas largas sin necesidad de mantener el ordenador personal encendido, se utilizó una **máquina virtual proporcionada por el mismo grupo de investigación**. Esta máquina virtual tenía las siguientes características: Debian GNU/Linux 13 (trixie), 4 vCPU Intel Xeon Silver 4214R a 2.40 GHz, 3.8 GiB de RAM y 60 GB de almacenamiento.

En esta última máquina virtual se alojó la base de datos vectorial de **Qdrant**, herramienta de código abierto que ha permitido la gestión de la base de datos vectorial utilizada para almacenar los vectores de memoria generados por Mem0.

3.2.3. Modelos de lenguaje

Para el desarrollo de este proyecto se han utilizado diferentes modelos de lenguaje, tanto locales como a través de API, con el fin de evaluar su rendimiento y capacidad de integración con el sistema desarrollado. Para esto se ha hecho uso de la herramienta de código abierto **Ollama**, que permite la ejecución de diferentes modelos de lenguaje en local, y de la API de **OpenAI**, que ha permitido la integración de los modelos de lenguaje de OpenAI en el sistema desarrollado.

Los modelos de lenguaje utilizados en este proyecto han sido los siguientes:

- **llama3.3:latest**: Modelo de 70b de parámetros, desarrollado por Meta. Es un modelo con un gran rendimiento y, por tanto, ha sido usado como modelo de referencia para las pruebas, se ha usado como modelo local para Mem0 y ha sido, a través de todo el proyecto, el modelo evaluador de la *precisión* de los modelos de lenguaje.
- **qwen2.5:72b**: Modelo de 72b de parámetros, desarrollado por Alibaba. Modelo con un mayor número de parámetros que llama3.3, para poder evaluar el impacto de

un modelo más grande en el rendimiento del sistema desarrollado.

- **qwen3:32b**: Modelo de 32b de parámetros, desarrollado por Alibaba. Modelo con un menor número de parámetros que llama3.3, para poder evaluar el impacto de un modelo más pequeño en el rendimiento del sistema desarrollado.
- **mistral:8x22b**: Este modelo se eligió a posteriori para evaluar la pérdida de rendimiento ante un exceso en la ventana de contexto. Mientras que los modelos qwen solo permiten una entrada de 32k tokens, demasiado pequeña para la entrada de LongMemEval-S; y llama3.3 permite entradas de hasta 128k tokens, cabiendo perfectamente cada pregunta de LongMemEval-S; este modelo permite entradas de hasta 64k tokens, lo que lo hace ideal para evaluar el impacto de un contexto superior a la capacidad de la ventana.
- **gpt-5 nano**: Los modelos GPT-5 son los últimos modelos desarrollados por OpenAI a fecha de la redacción de este proyecto, y entre ellos se ha elegido el modelo *nano* por ser el más eficiente en cuestión de precio-rendimiento.

3.3. Metodología de trabajo

Para el desarrollo correcto del trabajo se han ido realizando una serie de reuniones periódicas con el tutor. Estas reuniones, llevadas a cabo semanalmente, han permitido ir marcando los objetivos a corto plazo y asegurar que el desarrollo del proyecto se mantenga dentro de los objetivos planteados. A continuación se detalla, de manera general y por quincenas, el desarrollo del proyecto:

- **15/09/2025 - 28/09/2025**: Durante esta primera quincena se llevaron a cabo las reuniones iniciales para establecer el foco y los objetivos del proyecto, estableciendo Mem0 como el sistema base de memoria a utilizar. Además, se revisaron los papers fundamentales a seguir para el desarrollo del proyecto, incluyendo los de Mem0 [2] y LongMemEval [14].
- **29/09/2025 - 12/10/2025**: Durante esta segunda quincena se llevó a cabo la instalación inicial de las herramientas del proyecto, haciendo pruebas aisladas de los diferentes sistemas: Ollama, Mem0, Qdrant, etc. Además, se comenzó a trabajar en la integración de Mem0 con Ollama, para poder usar los modelos de lenguaje locales con el sistema de memoria.
- **13/10/2025 - 26/10/2025**: Esta tercera quincena fue la más productiva en cuanto al desarrollo del código del proyecto, ya que se construyeron todas las abstracciones necesarias para el desarrollo de la pipeline de evaluación, y se hizo una implementación inicial de estas con los sistemas previamente mencionados.
- **27/10/2025 - 09/11/2025**: En esta cuarta quincena se instaló y configuró la base de datos vectorial de Qdrant en la máquina virtual, y se integró esta base de datos con el sistema de memoria de Mem0. Además, se creó un evaluador de *precisión* usando llama3.3, y se hicieron las primeras pruebas de rendimiento con el dataset LongMemEval-Oracle.

- **10/11/2025 - 23/11/2025:** Durante esta quinta quincena se llevaron a cabo las pruebas de rendimiento con el dataset Longmemeval-S, usando los diferentes modelos de lenguaje mencionados anteriormente. Además, se comenzaron a analizar los resultados obtenidos en estas pruebas.
- **24/11/2025 - 18/1/2026:** Durante el mes de diciembre y parte de enero se hizo un parón en el desarrollo debido a los exámenes y las vacaciones de Navidad, retomando el proyecto a finales de enero.
- **19/1/2026 - 1/2/2026:** Durante esta sexta quincena se retomó el proyecto, se añadieron nuevas métricas de rendimiento a la pipeline, se realizaron pruebas usando ya el dataset LongMemEval-S y se comenzó a escribir el informe del proyecto.

Diseño y resolución

En este capítulo se realiza la descripción de la implementación de...

4.1. Pipeline de evaluación

Con el fin de crear un entorno de evaluación lo más extensible posible, se decidió crear un pipeline de procesamiento de datos modular, de manera que las diferentes partes que lo componen puedan ser fácilmente intercambiables (véase la Figura 4.1).

Para ello, se crearon una serie de clases abstractas que definen la interfaz de cada una de las partes del pipeline, y una clase principal *Pipeline*, que se encarga de gestionar el flujo de datos entre dichas partes. Las clases principales son las siguientes:

- La clase *DatasetReader*, que se encarga de acceder a las preguntas del dataset, adaptándose a su estructura concreta, y las devuelve en un formato común establecido por la clase *Pregunta*.
- La clase *MemorySystem*, que representa la lógica de procesamiento de las preguntas, iterando sobre los objetos de tipo *Pregunta* generados en el paso anterior y aplicando dos funciones: una de ellas se encarga de procesar el contexto de la pregunta y la otra de responder a la pregunta utilizando dicho procesado, devolviendo un objeto de tipo *Respuesta*. Aunque la clase se llame *MemorySystem*, heredando de ella se han implementado los dos escenarios vistos en la sección 2.3.1, es decir, tanto el escenario de memoria como el de contexto completo.
- La clase *Evaluator*, encargada de evaluar alguna de las métricas explicadas en la sección 2.5, recibe tanto los objetos *Pregunta* como *Respuesta* y utiliza la información de ambos para calcular la métrica correspondiente. Aunque aquí se hable de la clase *Evaluator*, lo normal es que se creen varias clases que hereden de esta, cada una de ellas encargada de calcular una métrica concreta, y el pipeline se encarga de ejecutarlas todas.

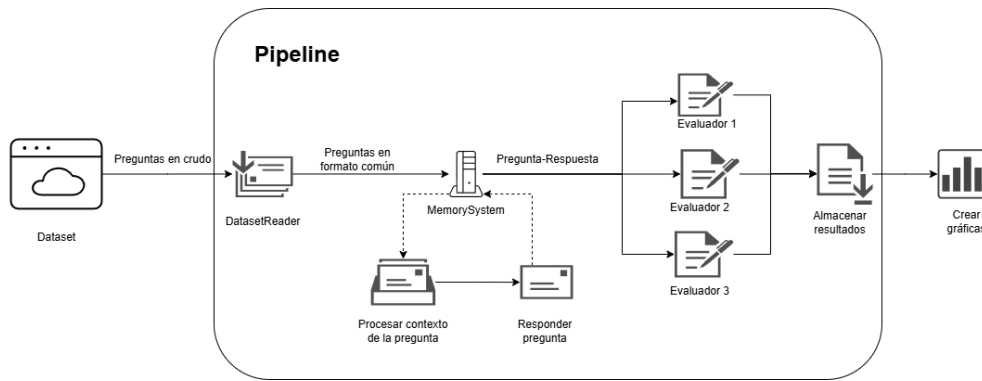


Figura 4.1: Esquema del pipeline de evaluación diseñado para este proyecto.

Cabe destacar que tanto la clase *MemorySystem* como la clase *Evaluator* pueden hacer uso de modelos de lenguaje para realizar su función, por lo que se ha creado una clase abstracta *LLMBackend* que define la interfaz de estos modelos, permitiendo trabajar de manera uniforme con diferentes API del mercado, como OpenAI, Ollama, etc.

Finalmente, el pipeline recibe la salida de los diferentes evaluadores y escribe los resultados de la evaluación en un archivo JSON. Este formato de salida permite una visualización sencilla de los resultados y facilita la comparación entre diferentes configuraciones del sistema o distintos modelos de lenguaje. Además, su estructura permite una integración sencilla con scripts de generación de gráficos o tablas para la presentación de resultados.

4.1.1. Implementación concreta

Partiendo de esta base, se han implementado diferentes clases concretas para poder realizar la evaluación de los diferentes escenarios de uso presentados en la sección 2.3.1 y de las diferentes métricas explicadas en la sección 2.5. En particular, se han implementado las siguientes clases concretas:

- *DatasetReader*:

1. *LongMemEvalReader*, el unico dataset utilizado en este proyecto es el Long-MemEval (vea la sección 2.4), por lo que solo se ha implementado un lector específico para este dataset, que se encarga de acceder a las preguntas y respuestas del mismo y adaptarlas al formato común establecido por la clase *Pregunta*.

- *MemorySystem*:

1. *FullContext*, clase simple que implementa el escenario de contexto completo, es decir, no realiza ningún tipo de procesamiento sobre el contexto de la pregunta, sino que simplemente lo pasa al modelo de lenguaje tal cual.
2. *Mem0_local*, clase que implementa el escenario de memoria, utilizando el sistema de memoria Mem0 (vea la sección 2.3.2), en particular su versión local, es decir, donde los datos de memoria se almacenan localmente.

3. *Mem0_api*, clase que implementa el escenario de memoria, utilizando el sistema de memoria Mem0, en particular su versión API, es decir, donde los datos de memoria se almacenan a través de una API externa.
- *Evaluator*:
1. *LLMJudgeEvaluator*, evaluador principal de exactitud basado en un modelo de lenguaje como juez. Compara respuesta generada y respuesta objetivo con una plantilla adaptada al tipo de pregunta, y decide si la respuesta es correcta.
 2. *F1ScoreEvaluator*, evaluador léxico que calcula precisión, *recall* y F1 a nivel de token entre la respuesta predicha y la respuesta de referencia.
 3. *LatencyEvaluator*, evaluador de rendimiento temporal durante la generación de respuesta, que reporta estadísticas agregadas como media, mediana y percentiles.
 4. *ContextProcessingTimeEvaluator*, evaluador del tiempo invertido en procesar el contexto antes de responder, útil para comparar el coste de preparación entre sistemas.
 5. *ReferenceAccuracyEvaluator*, evaluador orientado a la calidad de recuperación de memoria. Usa un juez LLM para estimar relevancia de memorias recuperadas y suficiencia del conjunto recuperado, y a partir de ello calcula precisión, *recall* y F1.
 6. *SessionPrecisionEvaluator*, evaluador de recuperación a nivel de sesión; mide si las memorias recuperadas provienen de las sesiones correctas respecto a las sesiones que contienen la respuesta.
 7. *TurnPrecisionEvaluator*, evaluador de recuperación a nivel de turno; mide si las memorias recuperadas corresponden a los turnos exactos donde está la información de respuesta.
 8. *MemoryTokenUsageEvaluator*, evaluador de eficiencia en uso de contexto, que cuantifica los tokens del *prompt* usado para responder y resume el consumo total y por tipo de pregunta.
 9. *MemoryAuditEvaluator*, evaluador de auditoría e inspección manual; para cada pregunta muestra las memorias recuperadas frente al conjunto total almacenado, facilitando análisis cualitativo del comportamiento del sistema de memoria.

4.2. Evaluación con LongMemEval-Oracle

La primera evaluación se realiza con el dataset LongMemEval-Oracle. Esta variante es la más ligera, ya que el contexto de cada pregunta se restringe a las sesiones relevantes para responderla. Con ello se evalúa el rendimiento de ambos escenarios en condiciones cercanas a las ideales, al minimizarse el ruido en el contexto.

En la Tabla 4.1 se resume estadísticamente el conjunto de conversaciones evaluado. Los datos corroboran lo anterior: el contexto por pregunta se limita a pocas sesiones, lo que se refleja en un número moderado de turnos y tokens. Aun así, cabe destacar la elevada variabilidad entre preguntas, lo que permite evaluar el rendimiento en condiciones diversas y no únicamente en casos triviales.

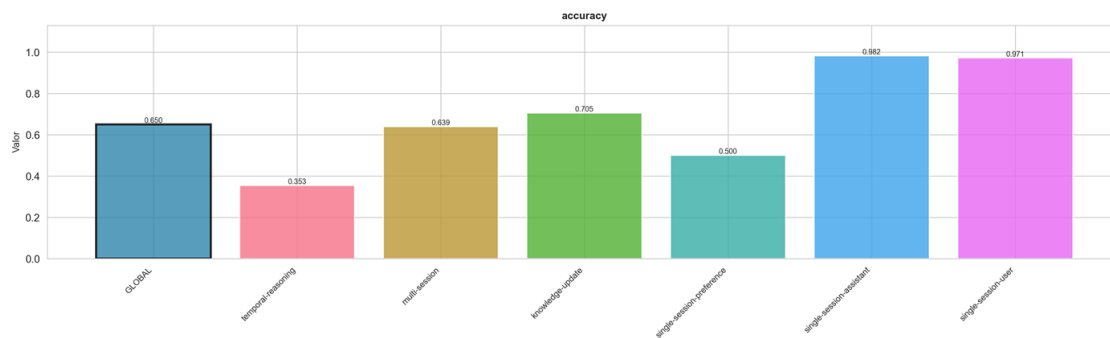
Métrica	Min	Max	Media	Mediana
Sesiones	1	6	1,90	2
Turnos	2	72	21,92	24
Tokens	110	22.098	5.639,21	5.661

Cuadro 4.1: Resumen estadístico del conjunto de conversaciones evaluado.

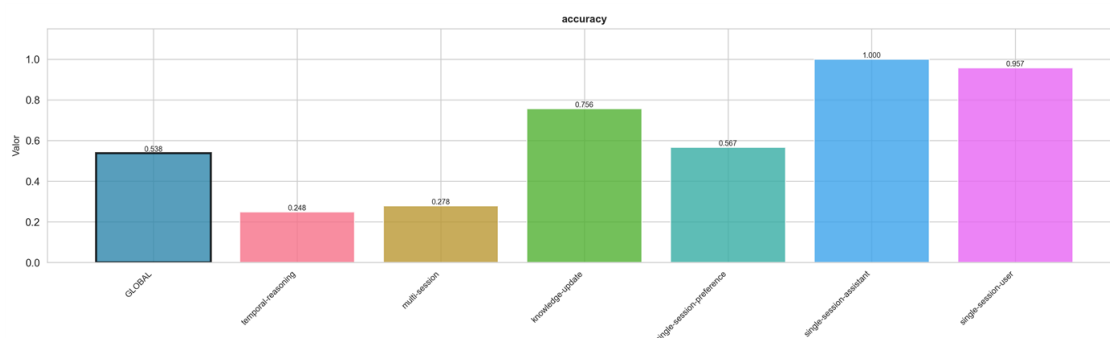
En este dataset es razonable esperar un rendimiento alto del escenario de contexto completo, puesto que el contexto de cada pregunta se ha construido para ser suficiente y, además, evita información irrelevante que pudiera degradar la respuesta. En consecuencia, el modelo debería poder responder correctamente sin necesidad de procesamiento adicional del contexto. Por su parte, el escenario de memoria también debería obtener buenos resultados, aunque podría no igualar al escenario de contexto completo en este entorno “limpio”.

A continuación se presentan los resultados para los dos escenarios considerados. En el escenario de contexto completo (FullContext) se han ejecutado tres modelos distintos: Llama3.3, Mistral y un modelo de OpenAI (GPT-5-nano). Dado que este enfoque depende fuertemente del modelo base, disponer de variedad resulta informativo. En el escenario de memoria (Mem0) se ha ejecutado únicamente Llama3.3, puesto que el objetivo del proyecto no es comparar modelos de lenguaje, sino comparar escenarios de uso.

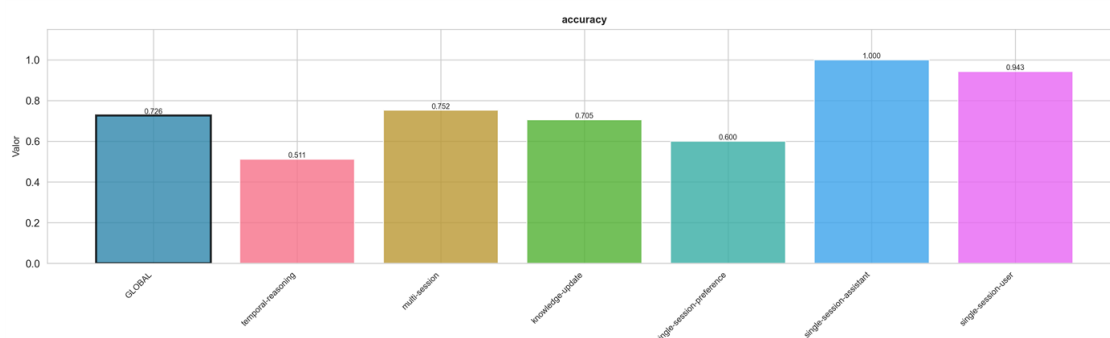
En la Figura 4.2 se compara la precisión estimada mediante un evaluador LLM-Judge. Además del resultado global, se muestran los valores desglosados por tipo de pregunta, lo que permite un análisis más detallado.



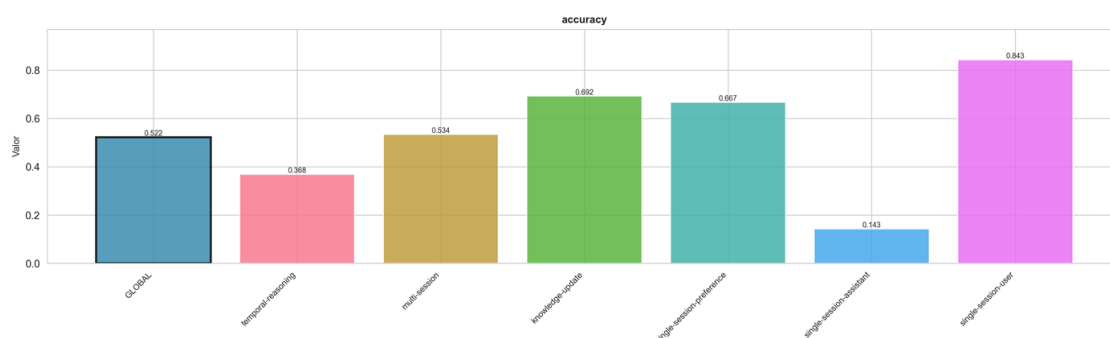
(a) FullContext-Llama3.3



(b) FullContext-Mistral



(c) FullContext-OpenAI



(d) Mem0-Llama3.3

Figura 4.2: Comparativa de la métrica de precisión (LLM-Judge).

Atendiendo únicamente al resultado global, dentro del escenario de contexto completo se observa que un modelo más potente mejora el rendimiento de forma apreciable (del orden del 10 % entre modelos). El escenario con Mem0 presenta el valor más bajo (52,2 %). Por tanto, si se considera solo esta métrica y bajo las condiciones de LongMemEval-Oracle, el escenario de contexto completo parece la opción más precisa.

No obstante, el desglose por tipo de pregunta permite interpretar mejor el comportamiento observado. En el escenario de contexto completo, la mayor degradación aparece en *temporal-reasoning*, que exige razonar sobre la secuencia temporal de eventos y sus relaciones. En el escenario de memoria, el rendimiento es relativamente homogéneo en la mayoría de categorías; sin embargo, *single-session-assistant* cae de forma marcada, lo que explica buena parte del descenso global. Una explicación plausible es que Mem0 tiende a no almacenar con la misma fidelidad información contenida en respuestas del asistente, precisamente la que este tipo de preguntas requiere.

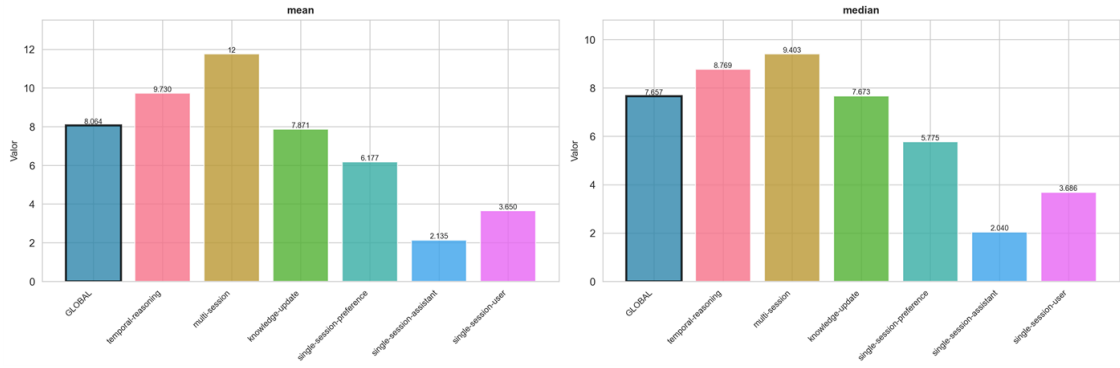
En síntesis, para contextos relativamente pequeños y depurados (como en LongMemEval-Oracle), el escenario de contexto completo ofrece la mayor precisión, al proporcionar al modelo toda la información necesaria sin pérdida por recuperación. Aun así, salvo en preguntas de tipo *single-session-assistant*, el escenario de memoria puede ser una alternativa razonable: aunque su precisión es inferior, no resulta necesariamente descartable y puede compensar cuando se prioriza la eficiencia en el uso de contexto.

A continuación se analizan el resto de métricas. Más adelante, al pasar a la siguiente versión del dataset (con contextos sustancialmente más largos), será posible extraer conclusiones sobre cómo escala cada escenario de uso.

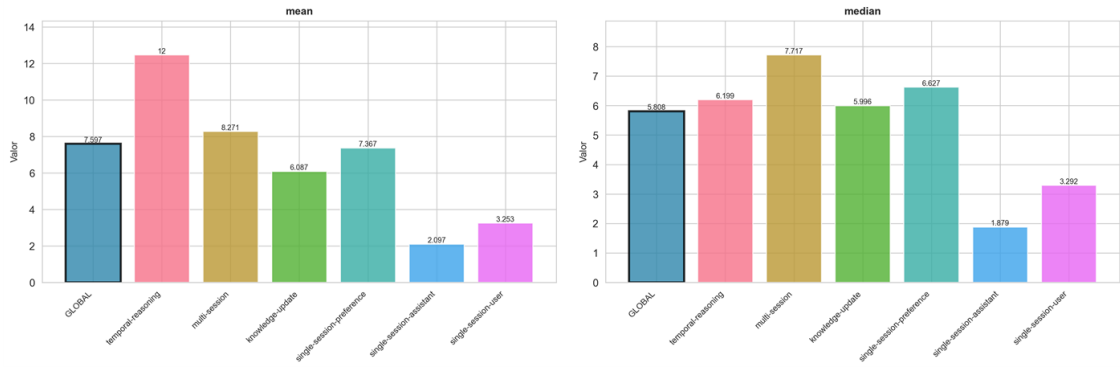
La Figura 4.3 compara la latencia, entendida como el tiempo transcurrido desde que se recibe la pregunta hasta que se genera la respuesta. Esta métrica es especialmente relevante en sistemas en tiempo real y es donde el escenario de memoria comienza a mostrar ventajas. Además, es razonable esperar que dicha ventaja aumente en la siguiente versión del dataset, al crecer el contexto por pregunta: el escenario de contexto completo deberá procesar más tokens en cada consulta, mientras que el escenario de memoria recuperará solo la información relevante.

Los resultados se presentan con dos desgloses. En primer lugar, se reportan media y mediana; en todos los casos la media es ligeramente superior, previsiblemente por la variabilidad observada, por lo que el análisis se apoya principalmente en la mediana. En segundo lugar, se incluye tanto el valor global como el desglose por tipo de pregunta, lo que permite una interpretación más rica.

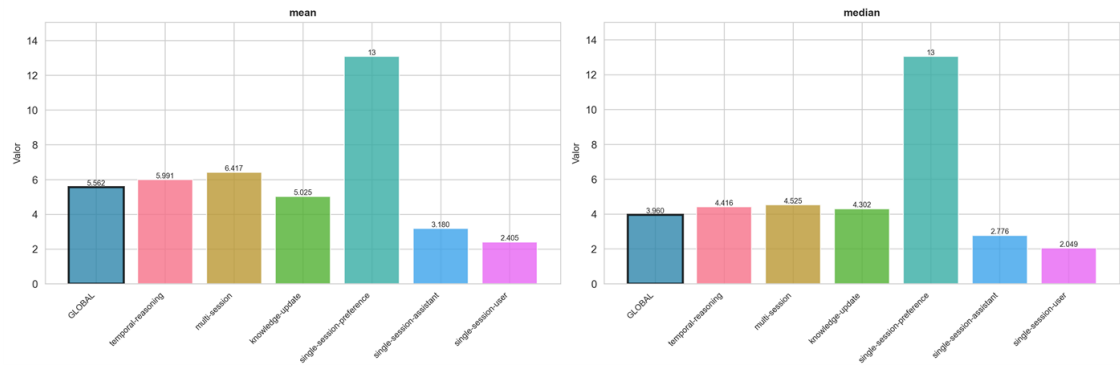
En el resultado global, el escenario de memoria presenta menor latencia que todos los escenarios de contexto completo, y es 3,48 veces más rápido que el escenario de contexto completo con el modelo de OpenAI (el más rápido dentro de FullContext). En consecuencia, bajo LongMemEval-Oracle, el escenario de memoria es el que ofrece mejor rendimiento en latencia.



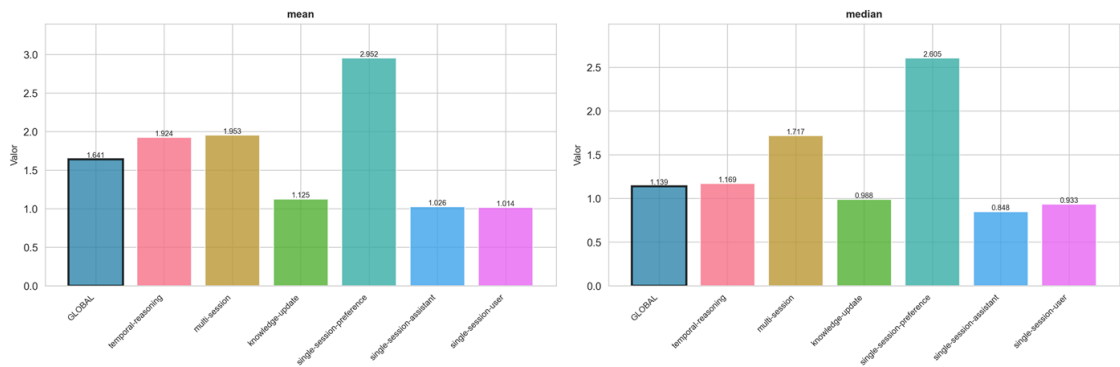
(a) FullContext-Llama3.3



(b) FullContext-Mistral



(c) FullContext-OpenAI



(d) Mem0-Llama3.3

Figura 4.3: Comparativa de la métrica de latencia.

Por tipo de pregunta, se observan patrones similares en FullContext con Llama3.3 y Mistral: salvo *single-session-assistant* y *single-session-user*, que tienden a resolverse algo más rápido, el resto de categorías presentan latencias parecidas. En cambio, el escenario de OpenAI y el de Mem0 muestran una distribución más uniforme y baja, con la excepción de *temporal-reasoning*, cuya latencia es notablemente superior. Este comportamiento sugiere que estas preguntas requieren mayor esfuerzo de razonamiento; cuando el modelo base es más capaz (OpenAI) o el contexto recuperado es más focalizado (Mem0), ese esfuerzo adicional puede manifestarse como un incremento del tiempo de generación.

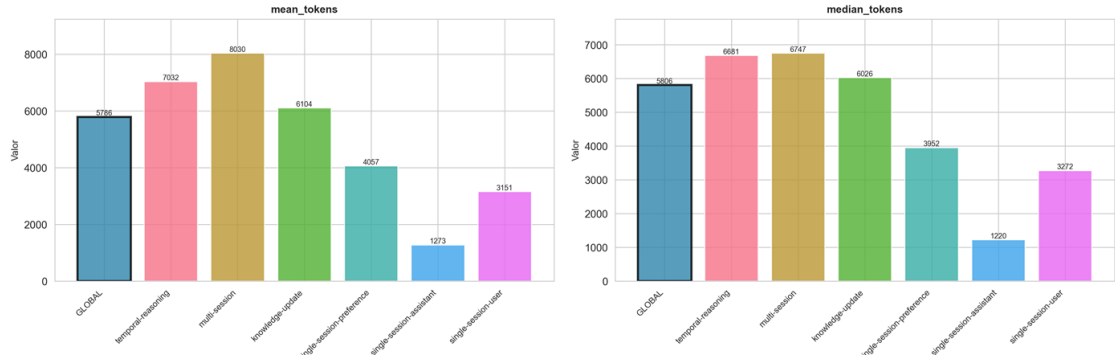
En conjunto, el desglose por tipo no introduce hallazgos cualitativamente distintos del resultado global: Mem0 mantiene una ventaja consistente en latencia. Es razonable anticipar que dicha ventaja se mantenga, e incluso aumente, en la siguiente versión del dataset, donde el contexto de cada pregunta es mayor.

En tercer lugar, se analiza la métrica de uso de tokens de contexto (Figura 4.4), que cuantifica cuántos tokens del contexto se utilizan para responder cada pregunta. Esta métrica evalúa la eficiencia en el uso del contexto y es donde el escenario de memoria muestra una ventaja especialmente marcada, al recuperar únicamente la información relevante.

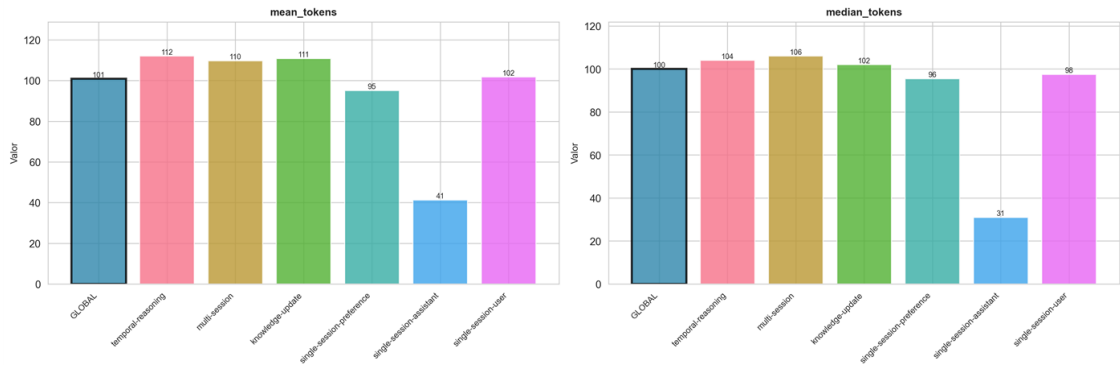
Al igual que en la métrica anterior, se reportan media y mediana. En este caso se muestra un único modelo para el escenario de contexto completo (Llama3.3), ya que el *prompt* sigue una plantilla fija que inserta la pregunta y el contexto sin procesarlo; por tanto, el número de tokens de contexto es esencialmente el mismo para todos los modelos en FullContext.

En esta métrica se observa una diferencia clara entre escenarios: la media de tokens de contexto en el escenario de contexto completo es 5786, mientras que en memoria es 101. Esto implica que Mem0 utiliza un 98,25 % menos de tokens de contexto para responder. No obstante, esta cifra debe interpretarse con cautela: aunque durante la generación se consuman pocos tokens, el sistema de memoria necesita procesar el contexto para recuperar información relevante, lo que puede introducir un coste adicional en tiempo y recursos. En consecuencia, la ventaja es especialmente significativa cuando se formulan varias preguntas sobre un mismo contexto, ya que el coste de procesamiento se amortiza entre consultas; en FullContext, en cambio, cada pregunta vuelve a incorporar y procesar el contexto completo, lo que puede resultar costoso cuando el contexto crece.

Una última métrica a considerar es el tiempo de procesamiento del contexto en el escenario Mem0, es decir, el tiempo invertido en procesar el contexto para crear las memorias antes de responder. Este coste no está incluido en la latencia reportada anteriormente (que considera el tiempo de respuesta una vez que el sistema ya dispone de memorias), pero resulta relevante para estimar el coste total del enfoque y realizar una comparativa honesta frente al escenario FullContext.



(a) FullContext-Llama3.3



(b) Mem0-Llama3.3

Figura 4.4: Comparativa de la métrica de uso de tokens en memoria.

Como se observa en la Figura 4.5, el tiempo de procesamiento del contexto presenta, en términos globales, una media de 230 s y una mediana de 212 s. Estos valores son superiores a la latencia de generación de respuesta en FullContext (en torno a 8 s en el peor caso de los evaluados), lo que implica que, para consultas aisladas, el escenario de memoria es considerablemente más lento. No obstante, esta inversión inicial puede amortizarse cuando se formulan varias preguntas sobre el mismo contexto, de modo que el escenario de memoria puede llegar a ser competitivo, e incluso ventajoso, en escenarios con múltiples consultas.

En particular, puede hacerse una estimación simple tomando como referencia la mediana de latencia de cada escenario (3,960 s para FullContext-OpenAI y 1,139 s para Mem0) y añadiendo al segundo un coste inicial fijo de 212 s por el procesamiento del contexto. Bajo esa aproximación, Mem0 pasa a ser más rápido a partir de la consulta número 76, ya que se cumple $212 + n \cdot 1,139 < n \cdot 3,960$ para $n > 75,15$. Este resultado sugiere que, aunque para preguntas individuales el escenario de memoria sea más lento, en situaciones con múltiples preguntas sobre un mismo contexto puede llegar a ser más eficiente que FullContext.

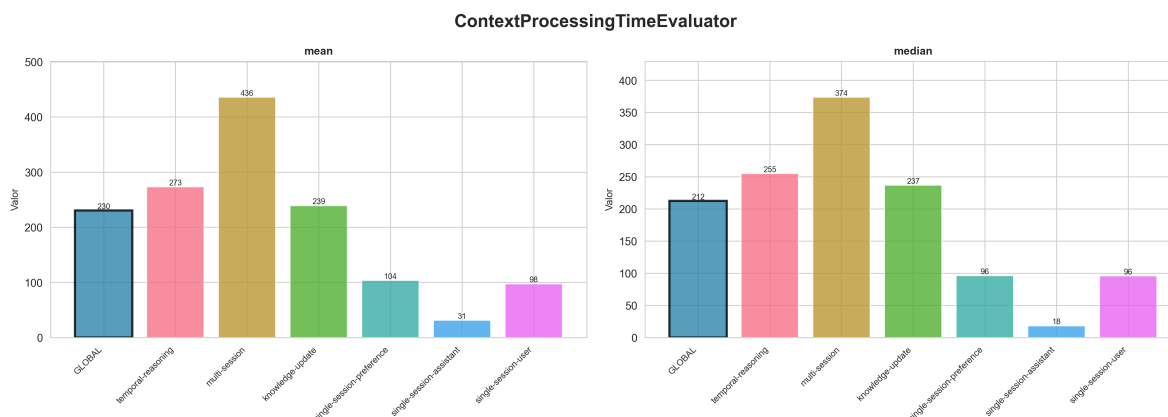


Figura 4.5: Tiempo de procesamiento del contexto en Mem0 (LongMemEval-Oracle).

4.3. Evaluación con LongMemEval-S

Aquí haría la comparativa de arriba, pero con las preguntas de longmemeval-s, que tienen un contexto mucho más largo. Y pondría gráficas enteras para comparar, p99 de uno con mediana del otro, etc.

Además, aquí al hablar de la precisión, es cuando menciono que mientras en el oracle mistral funciona bien en fullcontext, aquí pega un bajón porque el contexto deja de caber en la ventana de contexto.

4.4. Posibles mejoras del sistema de memoria

Aquí, iría el análisis sobre los posibles cambios en el sistema de memoria: embedder, motivado por el evaluador que recupera las memorias; modelos más potentes, o incluso menos potentes pero que son más económicos, etc.

También, si queda espacio, puedo hacer las pruebas de cambiar el prompt, y por ejemplo ver si suben las preguntas que fallan en single-session-assistant.

4.5. C02

No sé, si meter esto aquí como una sección aparte, o en el estado del arte hablar de como vamos a hacer el cálculo del C02: el cálculo era usando el tiempo (de respuesta para ambos escenarios, aunque en mem0 es insignificante) y el tiempo de procesamiento del contexto para mem0, y luego usar la potencia de la GPU del servidor y factor de utilización.

Te dejo un link a la conversación que tuve con Gemini, los dos últimos mensajes son los más claros: <https://gemini.google.com/share/b8800d143a00>

CAPÍTULO 5

Conclusiones y vías futuras

Finalizamos el trabajo estableciendo las conclusiones y vías futuras...

Conclusión sobre el impacto de la IA, con ventanas de contexto tan grandes, sistemas de memoria, y capacidades de razonamiento avanzado.

Bibliografía

- [1] The expanding horizon: Assessing the impact of extremely large context windows on retrieval-augmented generation systems in language models. *TIJER - International Research Journal*, 12(6):a582–a592, June 2025. URL: <https://tijer.org/tijer/papers/TIJER2506066.pdf>.
- [2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [3] Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts llm performance. Technical report, Chroma, July 2025. URL: <https://research.trychroma.com/context-rot>.
- [4] Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Peter Jansen, Oyvind Tafjord, Niket Tandon, Li Zhang, Chris Callison-Burch, and Peter Clark. Clin: A continually learning language agent for rapid task adaptation and generalization. *arXiv preprint arXiv:2310.10134*, 2023.
- [5] Kyle Montgomery, David Park, Jianhong Tu, Michael Bendersky, Beliz Gunel, Dawn Song, and Chenguang Wang. Predicting task performance with context-aware scaling laws. *arXiv preprint arXiv:2510.14919*, 2025.
- [6] Arvind Neelakantan, Tao Xu, Raul Puri, Alec Radford, Jesse Michael Han, Jerry Tworek, Qiming Yuan, Nikolas Tezak, Jong Wook Kim, Chris Hallacy, et al. Text and code embeddings by contrastive pre-training. arxiv 2022. *arXiv preprint arXiv:2201.10005*, 2022.
- [7] OpenAI. Tokenizer. <https://platform.openai.com/tokenizer>, 2024. Accedido: 13-02-2026.
- [8] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. Memgpt: Towards llms as operating systems. 2023.
- [9] Xiaodong Qu, Andrews Damoah, Joshua Sherwood, Peiyan Liu, Christian Shun Jin, Lulu Chen, Minjie Shen, Nawwaf Aleisa, Zeyuan Hou, Chenyu Zhang, et al. A comprehensive review of ai agents: Transforming possibilities in technology and beyond. *arXiv preprint arXiv:2508.11957*, 2025.

- [10] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- [11] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *International Conference on Machine Learning*, pages 31210–31227. PMLR, 2023.
- [12] Cole Stryker, Fangfang Lee, Dave Bergmann, and Mark Scapicchio. The 2026 Guide to Machine Learning. IBM Think, 2026. Accedido: 04-02-2026. URL: <https://www.ibm.com/think/machine-learning>.
- [13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [14] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- [15] Dianxing Zhang, Wendong Li, Kani Song, Jiaye Lu, Gang Li, Liuchun Yang, and Sheng Li. Memory in large language models: Mechanisms, evaluation and evolution. *arXiv preprint arXiv:2509.18868*, 2025.
- [16] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents, 2024. URL: <https://arxiv.org/abs/2404.13501>, arXiv:2404.13501.
- [17] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.
- [18] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 19724–19731, 2024.